DAY 7 | PRE-READ

Keys, Guards and Helpers

A light read before Day 7. About 15 minutes, no code required.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Why read this?

So far, anyone could call your API and do anything. Today you add what every real application needs: shared helpers, a checkpoint around every request, a secure way to prove who someone is, and a habit of checking code for security problems, including code written by AI.

1. Asking for what you need

In a hospital, a surgeon does not fetch instruments; they ask and a nurse hands them over. In FastAPI, an endpoint can simply say "I need the database" or "I need the logged-in user", and FastAPI hands it over. This is called **dependency injection**. **Middleware** is the reception desk every request passes.

2. Who are you, and what may you do?

Question	Name	Office example	Error if it fails
Who are you?	Authentication	Showing your ID at reception	401
What may you do?	Authorisation	Your badge opens some doors, not all	403

3. Never keep passwords in a drawer

A responsible company never stores your actual password. It stores a scrambled fingerprint of it, called a **hash**. If the database is ever stolen, the real passwords are not in it.

4. A visitor pass that expires NEW (v4)

After you log in, the server gives you a **token**: a visitor pass with your name, what you may do (its **scopes**) and an expiry time, stamped with a seal. A pass that lasts 15 minutes and opens only the doors you need does far less harm if someone copies it.

5. A list of the usual mistakes NEW (v4)

The **OWASP API Security Top 10** lists the most common ways APIs get attacked, such as reading someone else's data by changing an id in the URL. Security teams use it as a checklist. So will you.

6. Spell-check for security NEW (v4)

Tools like **bandit** and **pip-audit** read code and its list of libraries, and flag risky patterns and known vulnerable versions. You run them on everything, including code written by OpenAI Codex, and record what you find in a **vulnerability report**.

Key idea: code that works is not the same as code that is safe. Check both.

7. Think about these before class

- Why do websites ask you to log in again after some time?
- What could go wrong if changing /tickets/41 to /tickets/42 in a URL showed someone else's ticket?
- Why might a coding assistant suggest an old, unsafe pattern?

Anzen daiichi (安全第一): safety first